Anforderungen: KI-gestützte API-Discovery für Kernbanken-Schnittstellen

Ausgangslage und Problemstellung

Die Backend-Services des Projekts rufen Schnittstellen eines Kernbankensystems auf und stellen die Daten dem Frontend bereit. Bei jeder Neuanbindung oder Änderung liegen eine XSD und eine SHL-Schlüsseltabelle vor – das Verhalten des Service ist dennoch unbekannt.

Die XSD beschreibt Struktur, Datentypen und Kardinalitäten. Die SHL-Schlüsseltabelle enthält mögliche Ausprägungen. Ein Mapping, welches Element welches SHL-Schlüsselattribut verwendet, existiert jedoch nicht. Es muss aus dem fachlichen Kontext und den Antworten des Service hergeleitet werden.

Offen bleibt damit:

- welches SHL-Schlüsselattribut zu welchem Element gehört und welche Ausprägungen ein Element tatsächlich liefert oder akzeptiert,
- welche fachliche Bedeutung Elemente und Ausprägungen haben,
- welche Abhängigkeiten zwischen Attributen bestehen (z. B. „wenn A gesetzt ist, ist B Pflicht“),
- welche Elemente fachlich Pflicht sind, obwohl sie laut XSD optional sind – und umgekehrt,
- wie sich der Service bei fehlenden oder ungültigen Angaben verhält.

Bisher wird dies manuell durch Trial and Error ermittelt. Neben dem Bauen von Requests und dem Lesen von Fehlermeldungen besteht die Arbeit vor allem darin, zu beobachten, welche Ausprägungen die einzelnen Elemente zurückliefern. Diese werden dann über den fachlichen Kontext und die SHL-Schlüsseltabelle hergeleitet – ein Vorgehen, das bei jeder Anbindung viel Zeit kostet und am Wissen einzelner Personen hängt.

Zielsetzung und erwartete Ergebnisse

Ziel ist ein Prototyp, der aus XSD, SHL-Schlüsseltabelle, Beispiel-XML und dem Zugriff auf eine Testumgebung das Verhalten eines Service weitgehend automatisch erkundet. Daraus erzeugt er eine Dokumentation im bestehenden Format des Projekts.

Kern ist die fachliche Bedeutung. Für viele Elemente ist die Fachlichkeit unbekannt. Der Prototyp erschließt sie, indem er die zurückgelieferten Ausprägungen interpretiert, dem passenden SHL-Schlüsselattribut zuordnet und daraus Vorschläge zur Bedeutung von Element und Ausprägungen ableitet. KI wird vor allem hier eingesetzt, außerdem beim Formulieren und Prüfen von Hypothesen.

Der Prototyp ersetzt die manuelle Analyse nicht, sondern bereitet sie so weit vor, dass das Team nur noch prüft und ergänzt. Jede Aussage muss belegbar sein, denn eine falsche Dokumentation wäre irreführend.

Erwartete Ergebnisse:

1. Lauffähiger Prototyp mit Quellcode und Betriebsanleitung – zunächst für lesende Services (Teil 1), anschließend erweitert um schreibende Services (Teil 2).
2. Generierte Dokumentation für mindestens drei Referenz-Services je Teil.
3. Evaluation der generierten gegen manuell erstellte Dokumentation.
4. Schriftliche Ausarbeitung mit Methodik, Architektur und Grenzen des Ansatzes.
5. Übergabe an das Team inklusive Demo und Empfehlung zum Weiterbetrieb.

Scope und Aufteilung in zwei Teile

Das Vorhaben gliedert sich in zwei gleichermaßen wichtige Teile: Teil 1 entwickelt und erprobt den Ansatz für lesende Services, Teil 2 erweitert ihn auf schreibende Services.

	Teil 1: lesende Services	Teil 2: schreibende Services
Ziel	Ansatz entwickeln und erproben	Ansatz auf schreibende Operationen erweitern
Operationen	Nur lesend	Schreibend
Schwerpunkt	Ausprägungen in den Responses, SHL-Zuordnung, fachliche Bedeutung	Zusätzlich Seiteneffekte: was ein Aufruf verändert und wie sich das über lesende Services nachprüfen lässt

Für beide Teile gilt:

Im Scope	Nicht im Scope
XML-Services mit XSD	Produktivumgebung, Produktivdaten
Zuordnung der SHL-Schlüsselattribute und fachliche Bedeutung	Last- und Performancetests (jedoch Beachtung bei Services die intensive Operationen auf Seite des Kernbankensystems durchführen – es soll mit nicht performance-intensiven Services gestartet werden)
Struktur, Optionalität, Ausprägungen, Abhängigkeiten, Fehlerverhalten	Änderungen am Kernbankensystem

Eingangsartefakte

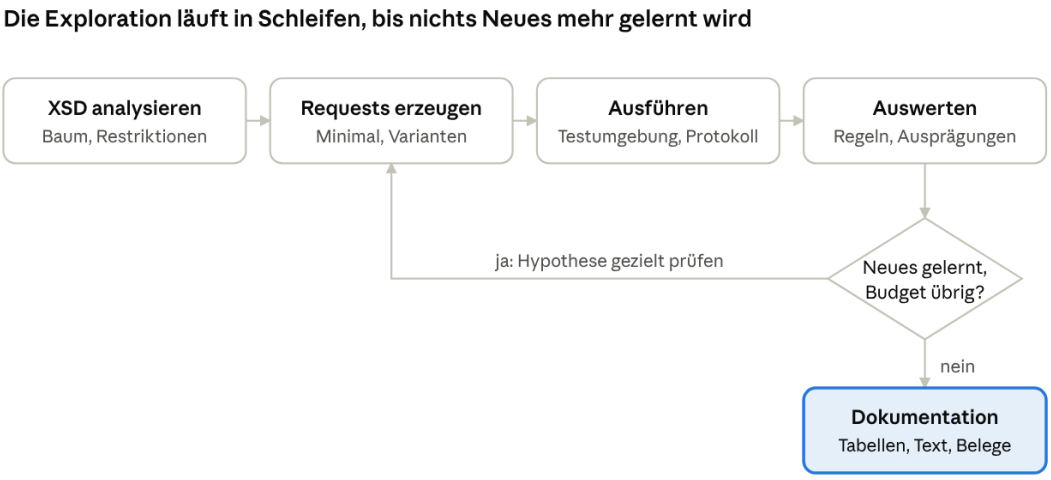
Vor Beginn der Implementierung erhält die bearbeitende Person vom Projekt die folgenden Artefakte.

Artefakt	Zweck
XSD (inkl. importierter Schemas)	Strukturanalyse, Request-Generierung

Artefakt	Zweck
SHL-Schlüsseltabelle	Mögliche Ausprägungen; Grundlage für die Zuordnung von Schlüsselattributen zu Elementen
Beispiel-XML je Service (Request und Response)	Verständnis der Struktur, Ausgangspunkt für die Request-Generierung, Abgleich mit beobachteten Antworten
Endpunkt der Testumgebung + technischer User	Ausführen der Requests
Testdatenbestand mit bekannten IDs (Kunden, Depots)	Gültige Einstiegswerte für Requests
Bestehende manuelle Dokumentationen	Ground Truth für die Evaluation, Beispiele für das LLM
Fehlercode-Katalog (optional)	Interpretation von Fehlermeldungen
Fachglossar / Abkürzungsverzeichnis	Ableitung der fachlichen Bedeutung
Freigegebenes LLM inkl. Zugang	KI-gestützte Hypothesen und Semantik

Funktionale Anforderungen

Das Werkzeug arbeitet in einer Schleife: Artefakte analysieren, Requests erzeugen, ausführen, Antworten auswerten, Hypothesen bilden und gezielt prüfen – bis das Aufrufbudget erschöpft ist oder keine neuen Erkenntnisse mehr entstehen.



Discovery-Schleife · 5 Schritte, 1 Entscheidung

Die Schleife von „Auswerten“ zurück zu „Requests erzeugen“ ist der Kern: Aus einer Vermutung wird ein gezielter Test, statt blind zu variieren.

Priorität: Muss = Abnahmekriterium, Soll = erwartet, Kann = optional. Alle Anforderungen gelten für beide Teile, sofern nicht „Teil 2“ vermerkt ist.

A. Analyse der Eingangsartefakte

ID	Anforderung	Priorität
FA-01	XSD inkl. `import`/`include` einlesen und den vollständigen Elementbaum für Request und Response erzeugen: Name, Ebene, Parent, Attribute, Datentyp, Kardinalität.	Muss
FA-02	Restriktionen extrahieren: `enumeration`, `pattern`, Längen, Wertgrenzen, `totalDigits`/`fractionDigits`; Konstrukte `choice`/`sequence`/`all`, Typvererbung, abstrakte Typen, Substitution Groups.	Muss
FA-03	SHL-Schlüsseltabelle und Beispiel-XML einlesen; die Werte der Beispiel-XML als erste Beobachtungen übernehmen.	Muss

B. Request-Generierung

ID	Anforderung	Priorität
FA-04	Ausgehend von der Beispiel-XML einen minimalen, schemavaliden Request erzeugen (nur Pflichtelemente laut XSD).	Muss
FA-05	Systematisch variieren: einzelne optionale Elemente hinzunehmen oder weglassen, Grenzwerte, ungültige Werte, Kombinationen (z. B. paarweises Testen), um Abhängigkeiten sichtbar zu machen.	Muss
FA-06	Werte beschaffen aus: bekannten Testdaten, Werten früherer Responses (Verkettung, z. B. Depotnummer aus einem Stammdepot-Aufruf als Eingabe für einen Folgeservice), Ausprägungen der SHL-Schlüsseltabelle, XSD-konformen Zufallswerten und LLM-Vorschlägen für fachlich plausible Werte.	Muss
FA-07	Adaptiv explorieren: aus bisherigen Ergebnissen Hypothesen ableiten und gezielt Requests zur Bestätigung oder Widerlegung erzeugen; mit konfigurierbarem Abbruchkriterium.	Soll

C. Ausführung

ID	Anforderung	Priorität
FA-08	Requests gegen die Testumgebung ausführen und Request, Response, Status, Laufzeit und Fehler vollständig protokollieren.	Muss
FA-09	Rate-Limit, Aufrufbudget, Whitelist erlaubter Operationen und Not-Aus konfigurierbar machen. In Teil 1 sind nur lesende Operationen erlaubt,	Muss

ID	Anforderung	Priorität
	schreibende erst in Teil 2 nach expliziter Freigabe.	
FA-10	Blocker erkennen und den Lauf mit klarer Meldung anhalten, statt den Aufruf zu wiederholen.	Muss
FA-11	Teil 2: Seiteneffekte schreibender Aufrufe über lesende Services nachprüfen und die Testdaten nach dem Lauf zurücksetzen bzw. die Rücksetzung anstoßen.	Muss

D. Auswertung und Inferenz

ID	Anforderung	Priorität
FA-12	Fehlercodes und -texte sammeln, gruppieren und dem auslösenden Request zuordnen; daraus Validierungsregeln ableiten (z. B. „Feld X fehlt“ ⇒ X ist Pflicht).	Muss
FA-13	Tatsächliche Optionalität ermitteln: XSD-optional, aber fachlich Pflicht – und umgekehrt.	Muss
FA-14	Für jedes Element die beobachteten Ausprägungen mit Häufigkeit sammeln.	Muss
FA-15	Je Element das passende SHL-Schlüsselattribut zuordnen bzw. vorschlagen, indem die beobachteten Ausprägungen mit der SHL-Schlüsseltabelle abgeglichen werden; Werte ausweisen, die in keiner Schlüsseltabelle vorkommen.	Muss
FA-16	Abhängigkeiten erkennen: bedingte Pflicht, gegenseitiger Ausschluss, Wertabhängigkeit (A = „X“ ⇒ B ∈ {…}), Einfluss von Request-Parametern auf die Response-Struktur, gemeinsames Auftreten von Elementen in der Response.	Muss
FA-17	Jede abgeleitete Aussage mit Status (belegt / vermutet / widerlegt), Konfidenz und Verweis auf die belegenden Requests versehen.	Muss

E. Fachliche Bedeutung

ID	Anforderung	Priorität
FA-18	Fachliche Bedeutung von Elementen und Ausprägungen per LLM vorschlagen – auf Basis der beobachteten Ausprägungen, des zugeordneten SHL-Schlüsselattributs, des Elementnamens, des Glossars und des Kontexts; bis zur fachlichen Bestätigung als Vorschlag kennzeichnen.	Muss
FA-19	Korrekturen fließen in Glossar und Folgeläufe ein.	Soll

F. Dokumentation

ID	Anforderung	Priorität
FA-20	Tabellen für Request und Response im bestehenden Format erzeugen (siehe Abschnitt Zielformat).	Muss
FA-21	Fließtext erzeugen: Zweck des Service, Minimal-Request, typische Aufrufmuster, Verhalten bei fehlerhaften Requests.	Muss
FA-22	Ergebnisse maschinenlesbar (JSON) ablegen; Tabellen und Text daraus rendern; Export als Markdown und Excel.	Muss
FA-23	Zwei XSD-Versionen bzw. Discovery-Läufe vergleichen und die Unterschiede ausweisen.	Soll
FA-24	Beispiel-Requests und Mock-Stubs (z. B. WireMock) als Nebenprodukt erzeugen.	Kann

Zielformat der Dokumentation

Die generierte Dokumentation übernimmt die bestehende Tabellenstruktur des Projekts vollständig, für Request und Response gleichermaßen. Ergänzende Spalten machen das Discovery-Ergebnis nachvollziehbar; sie sind ein Vorschlag und können in der Ausgabe ausgeblendet werden.

Bestehende Struktur am Beispiel:

Name	Ebene	Parent Element	Attribut(e)	Datentyp	Kardinalität	Beispiel	Beschreibung
Stammdepot	1	-	-	-	0..n	n/a	Liste an Stammdepots des Kunden

Spalten und ihre Quelle:

Spalte	Quelle	Status
Name, Ebene, Parent Element, Attribut(e), Datentyp, Kardinalität	XSD (deterministisch)	Bestehend
Beispiel	Beispiel-XML bzw. beobachtete Responses	Bestehend
Beschreibung	LLM-Vorschlag auf Basis der Ausprägungen, fachlich bestätigt	Bestehend
SHL-Schlüsselattribut	Zuordnung über beobachtete Ausprägungen und SHL-Schlüsseltabelle; verlinkt auf die möglichen	Bestehend (bisher als

Spalte	Quelle	Status
	Ausprägungen	Verlinkung auf die SHL-Schlüsseltabelle)
Restriktionen	XSD (Enumeration, Pattern, Länge)	Neu
Pflicht laut XSD / Pflicht beobachtet	XSD bzw. Verhalten des Service	Neu
Abhängigkeiten	Inferenz, z. B. „Pflicht, wenn X = ‚Y‘“	Neu
Status / Konfidenz	belegt, vermutet, widerlegt, fachlich bestätigt	Neu

Evaluation und Abnahmekriterien

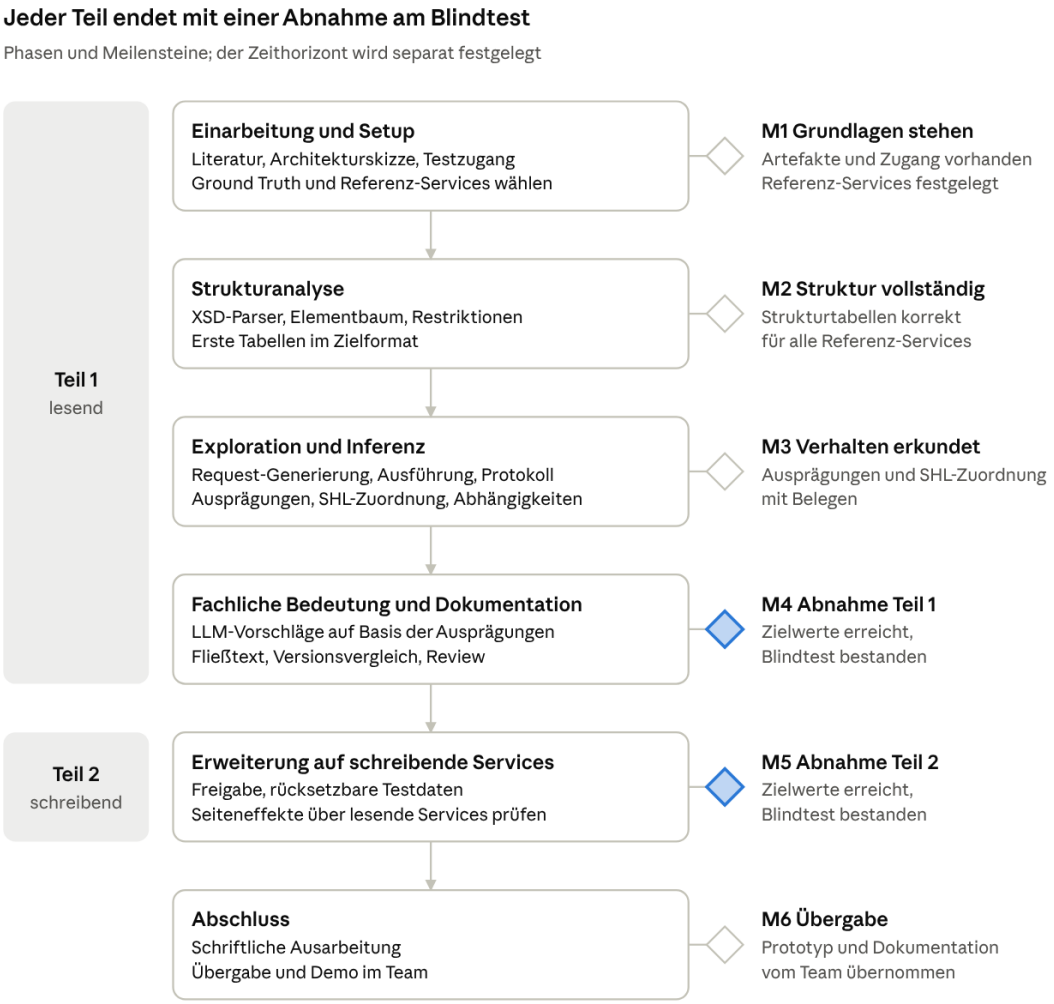
Jeder Teil wird an 3–5 Services gemessen, für die eine manuell erstellte Dokumentation als Ground Truth existiert, plus einem Service, den das Team noch nicht kennt (Blindtest). Die Zielwerte sind Vorschläge und werden zum Start gemeinsam festgelegt.

Kriterium	Messung	Zielwert (Vorschlag)
Fachliche Bedeutung	Bewertung durch den Fachbereich: korrekt / teilweise / falsch	≥ 70 % korrekt oder teilweise
Zuordnung SHL-Schlüsselattribut	Anteil der Elemente mit korrekt zugeordnetem Schlüsselattribut	≥ 90 %
Ausprägungen	Anteil bekannter Ausprägungen, die beobachtet wurden	≥ 80 %
Fachliche Pflichtfelder	Precision und Recall gegenüber Ground Truth	Recall ≥ 90 %
Abhängigkeiten	Precision und Recall der gefundenen Regeln	Precision ≥ 80 %
Strukturelle Vollständigkeit	Anteil korrekt erfasster Elemente inkl. Datentyp und Kardinalität	100 %
Unbelegte Aussagen	Aussagen ohne Verweis auf einen Request	0
Seiteneffekte (nur Teil 2)	Anteil korrekt nachgewiesener Änderungen je schreibendem Aufruf	≥ 90 %
Aufwand	Anzahl Aufrufe, Laufzeit, Prüfaufwand des Teams im Vergleich zur manuellen Analyse	Zeitersparnis ≥ 50 %

Abnahme: Je Teil erzeugt der Prototyp für den Blindtest-Service eine Dokumentation, die das Team nach Prüfung als Grundlage für die Anbindung einsetzen kann.

Phasen und Meilensteine

Das Vorhaben gliedert sich in sechs Phasen, jede endet mit einem Meilenstein, an dem gemeinsam über das weitere Vorgehen entschieden wird. Der Zeithorizont wird separat festgelegt.



Phasen und Meilensteine · 2 Teile, 6 Phasen

Die Strukturanalyse ist bewusst früh angesetzt: Sie ist deterministisch und liefert einen nutzbaren Mehrwert, bevor die KI-gestützten Teile fertig sind.

Als Einstieg in die Literatur eignen sich Arbeiten zu Specification Mining und dynamischer Invarianten-Erkennung (z. B. Daikon), zu automatisiertem API-Testing (z. B. RESTler, EvoMaster) sowie aktuelle Ansätze zu LLM-gestütztem API-Testing.